

Purge Git data

Overview

From time to time, a GitLabber may push a commit with data they didn't intend to commit/push to their repo on GitLab.com. The user may delete the branch if able, or rewrite their git history and force push, but other data may still be left dangling. In those cases, for confidentiality or security, waiting for an eventual garbage collection to get rid of such data may not be sufficient, and the following manual steps may need to be taken:

Checklist

- Delete Merge Requests. For example, if a security Merge Request was opened on GitLab.com instead of on dev.gitlab.org (as specified in our Security Releases documentation), it's important to ensure it's deleted to avoid out of time disclosure of vulnerabilities. Deleting Merge Requests can only be done by project owners or admins through the UI or the API
- Delete pipelines. CI/CD pipelines and builds may still retain data such as commit names. This can be done via the API (<https://docs.gitlab.com/ee/api/pipelines.html#delete-a-pipeline>)
- Trigger a full Garbage Collection run on the project. Unfortunately, manual housekeeping through the UI doesn't reliably trigger a full GC (see <https://gitlab.com/gitlab-com/gl-infra/reliability/-/issues/6960>), so you'll need to run the following in a production rails console, with the relevant `project_id`:
`Repositories::HousekeepingService.new(Project.find(project_id), :gc).execute`
- Check that the objects are gone. You may use for example `git cat-file -e <commit_id>` and check the return status.

If a full GC run doesn't delete the commits you can use the following, more aggressive steps by logging in to the file server that contains the repository:

NOTE: DO NOT RUN THE FOLLOWING COMMANDS ON A POOL REPOSITORY (i.e. make sure the repo path doesn't contain `/@pools/`. See https://gitlab.com/gitlab-org/gitlab-ce/merge_requests/29139/diffs)

- Manually delete the commits: `git -C <repo_path> show-ref | grep <commit_id>` and `git -C <repo_path> show-ref | grep <ref name>`, then `git -C <repo_path> update-ref -d <those refs>`
 - For example: `git -C /var/opt/gitlab/git-data/repositories/@hashed/63/64/63646ac9956a update-ref -d refs/keep-around/295a72cefa438c3ffa295dcaf92750c065fb9e59`
- Run an aggressive gc: `git -C <repo_path> -c gc.reflogExpire=0 -c gc.reflogExpireUnreachable=0 -c gc.rerereUnresolved=0 -c gc.rerereUnresolved=0 -c gc.pruneExpire=now gc` (source

<https://stackoverflow.com/questions/1904860/how-to-remove-unreferenced-blobs-from-my-git-repo>)

If after these steps the objects persist you may be dealing with a pooled repository, and further manual action is required. See https://gitlab.com/gitlab-org/gitlab-ce/blob/master/doc/development/git_object_deduplication.md for more information. Reach out to the development team for advice.